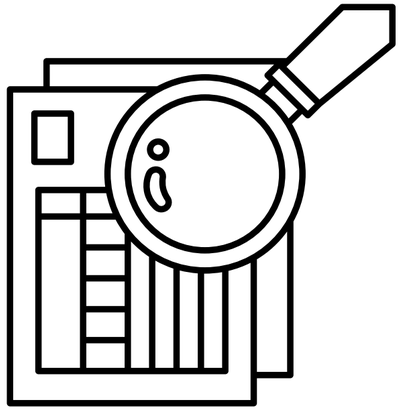
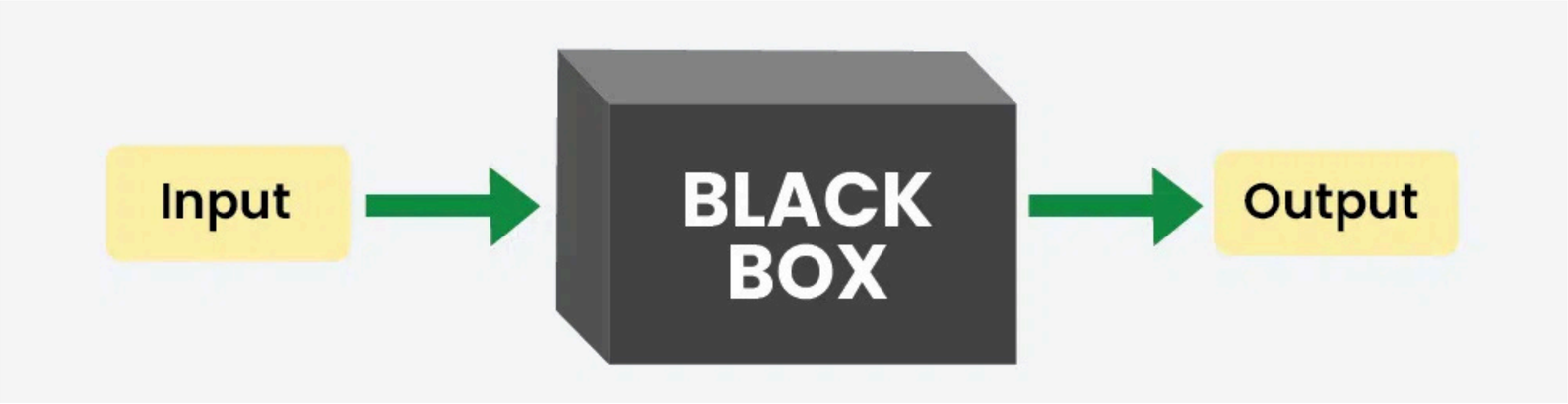
TESTING TYPES



BLACK-BOX TESTING

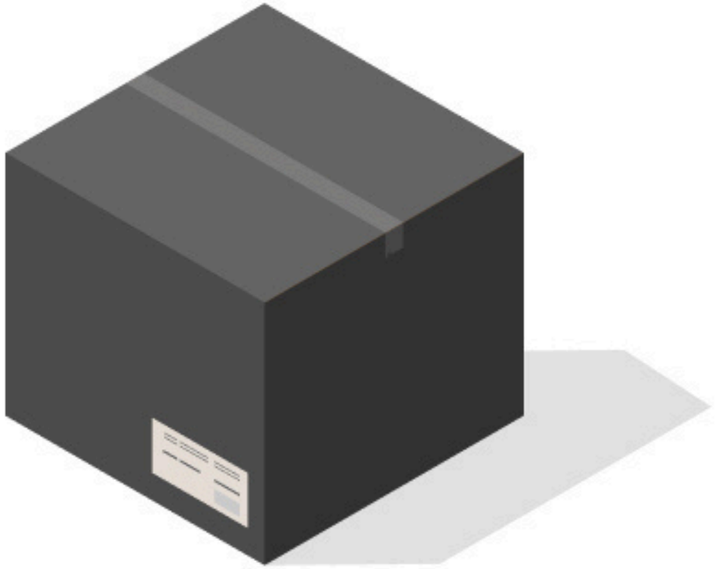
- Black-box testing is a functional testing technique where the tester verifies the system from the outside.
- The tester does not need to know or see the internal structure, logic, or code of the system.
- Instead, testing is done based on requirements, specifications, and user behavior.
- The main question answered is: “Does the system work as expected for the user?”
- It focuses on inputs and expected outputs.
- Black-box testing is very useful in system testing, acceptance testing, and user validation, because it simulates real-world usage.



WHITE-BOX TESTING

- White-box testing (also called structural testing) is the opposite approach. It requires knowledge of the internal code, logic, and structure of the system.
- The tester (often a developer) designs test cases by examining the code itself, making sure every branch, loop, and condition is executed at least once.
- The main question answered is: “Does the system’s code work correctly in all paths?”
- White-box tests often involve unit testing, integration testing, and security testing.
- It ensures things like:
  - Every if/else condition works correctly
  - Loops handle edge cases (0, 1, many iterations)
  - Exceptions are thrown and caught correctly
- This type of testing is critical in **safety-critical or financial systems** where even a hidden logic bug could cause serious harm.

QA testers



Black box - we do not know anything

Developers

White box - we know everything

BLACK-BOX VS WHITE-BOX TESTING

Aspect	Black-box Testing	White-box Testing
Definition	Testing from the **outside**; verifies functionality without looking at the code.	Testing from the **inside**; validates internal logic, branches, and code structure.
Knowledge needed	Only requirements, specifications, and user stories.	Detailed knowledge of the source code, design, and algorithms.
Focus	What the system **does** (behavior).	How the system **works** (logic and flow).
Who does it	QA testers, end users, acceptance testers.	Developers, SDETs, white-hat security testers.
Examples	Login with valid/invalid password, checkout totals, error messages.	Branch coverage, loop testing, exception handling, performance-critical code.
Advantages	Simple to apply; reflects user perspective; uncovers missing requirements.	High coverage of code paths; finds hidden logic bugs; good for optimization & security.
Limitations	Cannot guarantee internal code quality; might miss hidden bugs.	Cannot find missing requirements; time-consuming; requires coding knowledge.
When to use	System testing, acceptance testing, customer validation.	Unit testing, integration testing, financial/safety-critical modules.

CASE STUDY 1

A food delivery application has recently introduced a new feature that allows customers to schedule orders in advance. The feature promises that a user can choose items, set a delivery time (up to 48 hours later), and receive confirmation. However, customers have reported mixed experiences: sometimes the confirmation arrives late, sometimes discounts disappear, and occasionally the payment goes through but no receipt is issued. The company wants to ensure that customers always experience a smooth, reliable ordering flow without worrying about the system's internal workings.

WHICH APPROACH OF TESTING TO BE USED HERE?

CASE STUDY 1: SOLUTION

- This scenario is about validating the end-to-end behavior as experienced by the customer: order placement, scheduled delivery, discounts applied, confirmation receipt.
- The concern is whether the system matches the requirements, not how the code implements scheduling or payments.
- It will verify user-visible outcomes, e.g., “If I schedule at 7 PM for tomorrow, I should receive a confirmation instantly and delivery at the requested time.”

THE CORRECT APPROACH IS BLACK-BOX TESTING

CASE STUDY 2

An insurance company develops a module to calculate premium adjustments based on multiple conditions: age group, accident history, vehicle type, loyalty years, and seasonal promotions. The requirements document only states that the “system shall correctly calculate the customer’s adjusted premium.” During internal reviews, developers highlight that the logic has deeply nested conditions and several exception branches (e.g., customers with multiple discounts, drivers under 18, suspended policies). A small mistake in any branch could lead to incorrect billing or legal disputes.

WHICH APPROACH OF TESTING TO BE USED HERE?

CASE STUDY 2: SOLUTION

- On the surface, the requirement looks like a single function: “calculate premium.”
  A Black-box tester could try by trying a few input combinations.

- But because there are many hidden branches and nested rules, it’s impossible to cover all risks by treating it only as a black-box.

- To guarantee correctness, testers must dive into the internal code logic, covering every condition, ensuring loops and exceptions behave properly.

THE CORRECT APPROACH IS WHITE-BOX TESTING

TEST CASES

When we want to test a system, we don’t just test randomly.

We design test cases, which are step-by-step instructions describing:

- What we are testing (the requirement).
- What inputs we use.
- What steps we follow.
- What output we expect.

This ensures testing is organized, repeatable, and covers all scenarios.

Field	Description
ID	Unique identifier for the test case (e.g., TC-001).
Test Scenario	A short description of the situation being tested.
Test Case Title	What this test is verifying.
Preconditions	What must exist before the test (e.g., user account).
Steps	Actions to perform during the test.
Input / Data	The actual values used in the test.
Expected Result	What should happen if the system works correctly.
Actual Result	What actually happened (filled during execution).
Status	Pass/Fail (after execution).

FUNCTIONAL REQUIREMENTS

“The system shall allow a user to log in using email and password. If credentials are invalid, an error message shall be shown.”

ID	Test Scenario	Test Case Title	Preconditions	Steps	Input / Test Data	Expected Result
TC-LOGIN-001	Valid login	Login with correct email & password	User exists and is active	1) Open login page 2) Enter valid email & password 3) Submit	Email: user@example.com, Pass: Correct123	Dashboard/homepage is displayed
TC-LOGIN-002	Invalid password	Login attempt with wrong password	User exists	1) Enter valid email + wrong password 2) Submit	Email: user@example.com, Pass: Wrong123	Error message: “Invalid email or password.”
TC-LOGIN-003	Empty fields	Try login without entering credentials	None	1) Leave email and password blank → 2) Submit	Email: *(blank)*, Pass: *(blank)*	Error: “Email and password are required.”

THANK YOU